

[← Go Back](#)

Hardware

Portenta H7

Tutorials

[BLE Connectivity on Portenta H7](#)[Creating GUIs with LVGL \(V7\)](#)[Dual Core Processing](#)[Creating a Flash-Optimized Key-Value Store](#)[Getting Started with OpenMV and MicroPython](#)[Debugging with Lauterbach TRACE32 GDB Front-End Debugger](#)[Arduino Portenta H7 MicroPython Cheat Sheet](#)[Over-The-Air \(OTA\) Updates with the Arduino Portenta H7](#)[Reading and Writing Flash Memory](#)[Secure Boot on Portenta H7](#)[Setting Up Portenta H7 For Arduino](#)[Voice Commands With The Arduino Speech Recognition Engine](#)[Updating the Portenta Bootloader](#)[Portenta H7 as a USB Host](#)[Portenta H7 as a Wi-Fi Access Point](#)

Home / Hardware / Portenta H7 / [Creating a Flash-Optimized Key-Value Store](#)

Creating a Flash-Optimized Key-Value Store

This tutorial explains how to create a Flash-optimized key-value store using the Flash memory of the Portenta H7.

Author [Giampaolo Mancini, Sebastian Romero](#) · Last revision [01/25/2024](#)

ON THIS PAGE

Overview

[Goals](#) +[Instructions](#) +[Conclusion](#) +

Overview

This tutorial explains how to create a Flash-optimized key-value store using the Flash memory of the Portenta H7. It builds on top of the [Flash In-Application Programming](#) tutorial.

Goals

In this tutorial you will learn how to use the Mbed OS [TDBStore API](#) to create a [Key value store](#) in the free space of the microcontroller's internal Flash.

Required Hardware and Software

[Portenta H7 \(ABX00042\)](#),
[Portenta H7 Lite \(ABX00045\)](#)
or [Portenta H7 Lite Connected \(ABX00046\)](#)

[Help](#)

Instructions

MbedOS APIs for Flash Storage

The core software of Portenta H7 is based on the Mbed OS operating system, allowing developers to integrate the Arduino API with the APIs exposed by Mbed OS.

Mbed OS has a rich API for managing storage on different mediums, ranging from the small internal Flash memory of a microcontroller to external SecureDigital cards with large data storage space.

In this tutorial, you are going to save a value persistently inside the Flash memory. That allows to access that value even after a reset of the microcontroller. You will retrieve some information from a Flash block by using the [FlashIAPBlockDevice](#) and the [TDBStore](#) APIs. You will use the `FlashIAPBlockDevice` class to create a block device on the free space of the Flash and you will create a Key-Value Store in it using the `TDBStore` API.



Important: The TBSStore API optimizes for access speed, reduce [wearing of the flash](#) and minimize storage overhead. TBSStore is also resilient to power failures. If you want to use the Flash memory of the microcontroller, always prefer the TBSStore approach over a direct access to the FlashIAP block device.

1. The Basic Setup

Begin by plugging in your Portenta board to the computer using a USB-C® cable and open the Arduino IDE. If this is your first time running Arduino sketch files on the board, we suggest you check out how to [set up the Portenta H7 for Arduino](#) before you proceed.

2. Create the Structure of the Program

Let's program the Portenta with a sketch. You will also define a few helper functions in a supporting header file.

Create a new sketch named `FlashKeyValue.ino`

Create a new file named `FlashIAPLimits.h` to store the helper functions in a reusable file.

Note: Finished sketch its inside the tutorials library wrapper at:

**Examples > Arduino_Pro_Tutorials
> Creating a Flash-Optimized Key-Value Store > FlashKeyValueStore**

First let's add the helper functions to the `FlashIAPLimits.h` header. This will determine the available Flash limits to allocate the custom data.

```

1  /**
2  Helper functions for ca
3  **/
4
5  // Ensures that this fi
6  #pragma once
7
8  #include <Arduino.h>
9  #include <FlashIAP.h>
10 #include <FlashIAPBlock
11
12 using namespace mbed;
13
14 // A helper struct for
15 struct FlashIAPLimits {
16     size_t flash_size;
17     uint32_t start_address;
18     uint32_t available_size;
19 };
20
21 // Get the actual start
22 // considering the spac
23 FlashIAPLimits getFlash
24 {
25     // Alignment lambdas
26     auto align_down = [](
27         return ((val) / si
28     };

```

4. Make the Key Store Program

Go to `FlashKeyValue.ino` and include the libraries that you need from **MBED** and your header helper (`FlashIAPLimits.h`). The `getFlashIAPLimits()` function which is defined in the `FlashIAPLimits.h` header takes care of not overwriting data already stored on the Flash and aligns the

those calculated limits to create a block device and a `TDBStore` on top of them.

```
1 #include <FlashIAPBlockD
2 #include <TDBStore.h>
3
4 using namespace mbed;
5
6 // Get limits of the In
7 #include "FlashIAPLimits
8 auto iapLimits { getFlas
9
10 // Create a block device
11 FlashIAPBlockDevice bloc
12
13 // Create a key-value st
14 TDBStore store(&blockDev
15
16 // Dummy sketch stats da
17 struct SketchStats {
18     uint32_t startupTime;
19     uint32_t randomValue;
20     uint32_t runCount;
21 };
```

In the `setup()` function at the beginning you will have to wait until the Serial port is ready and then print some info about the FlashIAP block device (`blockDevice`).

```

1 void setup()
2 {
3   Serial.begin(115200);
4   while (!Serial);
5
6   // Wait for terminal
7   delay(1000);
8
9   Serial.println("FlashI
10
11   // Feed the random num
12   srand(micros());
13
14   // Initialize the flas
15   blockDevice.init();
16   Serial.print("FlashIAP
17   Serial.println(blockDe
18   Serial.print("FlashIAP
19   Serial.println(blockDe
20   Serial.print("FlashIAP
21   Serial.println(blockDe
22   Serial.print("FlashIAP
23   Serial.println(blockDe
24   // Deinitialize the de
25   blockDevice.deinit();

```

After that, initialize the TDBstore (our storage space), set the key for the store data (`stats`), initialize the value that you will save `runCount` and declare an object to fetch the previous values (`previousStats`).

```

1 // Initialize the key-va
2 Serial.print("Initiali
3 auto result = store.in
4 Serial.println(result
5 if (result != MBED_SUC
6   while (true); // Sto
7
8 // An example key name
9 const char statsKey[]
10
11 // Keep track of the n
12 uint32_t runCount { 0
13
14 // Previous stats
15 SketchStats previousSt

```

Now that you have everything ready, let's retrieve the previous values from the store and update the store with the new values.

```

1 // Get previous run sta
2 Serial.println("Retri
3 result = getSketchSta
4
5 if (result == MBED_SL
6 Serial.println("Pre
7 Serial.print("\tSta
8 Serial.println(prev
9 Serial.print("\tRar
10 Serial.println(prev
11 Serial.print("\tRun
12
13 Serial.println(prev
14
15 runCount = previous
16
17 } else if (result ==
18 Serial.println("No
19 } else {
20 Serial.println("Err
21 while (true);
22 }
23
24 // Update the stats a
25 SketchStats currentSt
26 result = setSketchSta
27
28 if (result == MBED_SL

```

To finish the sketch, create

`getSketchStats` and

`setSketchStats` functions at the

bottom of the sketch (after the

`setup()` and `loop()`).

The `getSketchStats` function tries to retrieve the stats values stored in the Flash using the key `key` and returns them via the `stats` pointer parameter. Our `SketchStats` data struct is very simple and has a fixed size. You can therefore deserialize the buffer with a simple `memcpy` .

The `setSketchStats` function stores the `stats` data and assigns the key `key` to it. The key will be created in the key-value store if it does not exist yet.

```
1 // Retrieve SketchStats
2 int getSketchStats(const char* key)
3 {
4     // Retrieve key-value
5     TDBStore::info_t info;
6     auto result = store.get(key, info);
7
8     if (result == MBED_ERROR_NOT_FOUND)
9         return result;
10
11     // Allocate space for
12     uint8_t buffer[info.size];
13     size_t actual_size;
14
15     // Get the value
16     result = store.get(key, buffer, &actual_size);
17     if (result != MBED_SUCCESS)
18         return result;
19
20     memcpy(stats, buffer, actual_size);
21     return result;
22 }
23
24 // Store a SketchStats
25 int setSketchStats(const char* key, SketchStats stats)
26 {
27     return store.set(key, stats);
28 }
```

5. Results

Upload the sketch, the output should be similar to the following:



```
1 FlashIAPBlockDevice + TD
2 FlashIAP block device si
3 FlashIAP block device re
4 FlashIAP block device pr
5 FlashIAP block device er
6 Initializing TDBStore: 0
7 Retrieving Sketch Stats
8 Previous Stats
9     Startup Time: 12
10    Random Value: 15
11    Run Count: 13
12 Sketch Stats updated
13 Current Stats
14    Startup Time: 42
15    Random Value: 21
16    Run Count: 14
```

 Note that the Flash memory will be **erased** when a new sketch is uploaded.

Push the reset button to restart the sketch. The values of the stats have been updated. `Previous Stats` which is retrieved from the key-value store now contains values from the previous execution.

Conclusion

You have learned how to use the available space in the Flash memory of the microcontroller to create a key-value store and use it to retrieve and store data.

It is not recommended to use the Flash of the microcontroller as the primary storage for data-intensive applications. It is best suited for read/write operations that are performed only once in a while such as storing and retrieving application configurations or persistent parameters.

Next Steps

Learn how to retrieve a collection of keys using TDBStore iterators via `iterator_open` and

`iterator_next`

Learn how to create an incremental TDBStore set sequence via `set_start`,

`set_add_data` and

`set_finalize`

Learn how to use the 16 MB QSPI Flash on the Portenta H7

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

